Git

Introduction

Imagine you are working on a **group project** with your classmates. Everyone writes code, but someone **accidentally deletes an important file**. How do you get it back? Or what if you make changes but later realize the old version was better?

This is where **Git helps**.

Section 1: Learn

What is Git?

Git is a version control system (VCS) that tracks changes in code, allowing multiple developers to work together **without overwriting each other's work**. It helps:

- **Save different versions** of your code.
- **Work collaboratively** with a team.
- **Go back to an earlier version** if needed.

Why is Git Important?

1. **Prevents data loss** – Keeps a history of code changes.
2. **Collaboration** – Multiple people can work on the same project.
3. **Reverts mistakes easily** – If something breaks, you can go back to a previous version.
4. **Works offline** – Unlike cloud-based tools, Git works on your local computer.

5. **Industry Standard** – Used by companies like **Google, Microsoft, Amazon, and Flipkart**.

How Does Git Work?

Git follows a simple process:

1. **Initialize a Repository** – Start tracking a project with Git.
2. **Stage Changes** – Select files to include in the next commit.
3. **Commit Changes** – Save the current state of the code.
4. **Push to Remote** – Upload changes to a cloud repository like **GitHub**.
5. **Pull Updates** – Download the latest code from the cloud.

An Interesting Anecdote

Before Git, developers used **manual file backups** or emailed files to each other. This led to **lost changes and confusion**. In 2005, **Linus Torvalds (the creator of Linux)** developed Git to manage **thousands of code changes** efficiently. Today, **Git is the most widely used version control system worldwide**.

Section 2: Practice

Step-by-Step Guide to Using Git

1. Setting Up Git

Before using Git, you need to **install and configure it**.

```
# Check if Git is installed
git --version


# Set up your user name and email
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

Why is this important?

- Ensures **every commit has your name attached**.
- Helps in tracking **who made which changes**.

---

2. Initializing a Git Repository

```
# Create a new project folder and navigate into it
mkdir my_project
cd my_project

# Initialize Git in the folder
git init
```

What happens here?

- Git starts **tracking all changes** in this folder.
- Creates a **hidden .git folder** to store history.

---

3. Adding and Committing Changes

```
# Create a new file
echo "print('Hello, Git!')" > script.py

# Add the file to Git staging
git add script.py

# Commit the changes
```

```
git commit -m "Added script.py file"
```

What does this do?

- git add **prepares the file** for commit.
- git commit **saves the file with a message** describing the change.

---

4. Pushing to GitHub (Remote Repository)

To save your code online, you need a **GitHub repository**.

```
# Connect local Git repository to GitHub
git remote add origin https://github.com/yourusername/my_project.git

# Push code to GitHub
git push -u origin main
```

Why is this useful?

- Keeps a **backup** of your code online.
- Allows your **team to access and collaborate** on the project.

---

5. Pulling Changes from GitHub

If someone else has updated the code, you need to **sync your local copy**.

```
# Pull the latest changes
git pull origin main
```

How does this help?

- Updates your **local copy** with the latest changes.
- Prevents **conflicts between team members**.

Real-World Example

A **software company** is developing a **mobile banking app**. Their **10-member team** works on different features like **login, payments, and security**.

Without Git:

- Team members **overwrite each other's work**.
- It is **hard to track who changed what**.
- If a bug appears, there is **no easy way to roll back**.

With Git:

1. Each team member **works on a separate branch**.
2. Changes are **tracked and saved** automatically.
3. If something goes wrong, they can **roll back to a stable version**.

Result? Faster development, **fewer errors, and better collaboration**.

Section 3: Know More

Frequently Asked Questions

1. **Is Git the same as GitHub?**

   No. **Git is a version control tool** used locally, while **GitHub is an online platform** to store and share Git repositories.
2. **Can I use Git without an internet connection?**

   Yes! Git works **offline**, but you need the internet to **push and pull changes from GitHub**.
3. **How do I undo changes in Git?**

```
# Undo the last commit (without deleting the changes)
git reset --soft HEAD~1
```

```
# Undo the last commit and delete the changes
git reset --hard HEAD~1
```

4. **What is the difference between Git and SVN?**
    - **Git** is **distributed**, meaning every developer has a full copy of the repository.
    - **SVN** is **centralized**, meaning there is only one central repository.
5. **Where can I practice Git?**
    - Create a **GitHub account** and start a sample project.
    - Try **committing and pushing changes** using Git.

Conclusion

Git is an **essential tool for every developer**. It helps teams **collaborate, track changes, and work efficiently**. By mastering Git, you can **manage code better and work on real-world projects smoothly**.

Start with **basic Git commands**, explore **GitHub**, and soon you'll be contributing to **big projects confidently**!